

AFRILANG Stdlib

std/c/graphcolor

Bibliothèque AFRILANG `std/c/graphcolor` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : greedyColor, c

```
`import "std/c/graphcolor" . `use graphcolor`
```

- `greedyColor(adj list number, n number) ? list number`
- `chromaticNumber(adj list number, n number) ? number`
- `isValidColoring(adj list number, n number, colors list number) ? bool`
- `colorClasses(adj list number, n number) ? list number`
- `conflictCount(adj list number, n number, colors list number) ? number`
- `welshPowell(adj list number, n number) ? list number`
- `sameColorNeighbors(adj list number, n number, node number) ? number`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphcolor` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : greedyColor, c

```
`import "std/c/graphcolor"` . `use graphcolor`  
- `greedyColor(adj list number, n number) ? list number`  
- `chromaticNumber(adj list number, n number) ? number`  
- `isValidColoring(adj list number, n number, colors list number) ? bool`  
- `colorClasses(adj list number, n number) ? list number`  
- `conflictCount(adj list number, n number, colors list number) ? number`  
- `welshPowell(adj list number, n number) ? list number`  
- `sameColorNeighbors(adj list number, n number, node number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphcolor"  
use graphcolor
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? greedyColor(adj list number, n number) ? list  
? chromaticNumber(adj list number, n number) ? number  
? isValidColoring(adj list number, n number, colors list number) ? bool  
? colorClasses(adj list number, n number) ? list  
? conflictCount(adj list number, n number, colors list number) ? number  
? welshPowell(adj list number, n number) ? list  
? sameColorNeighbors(adj list number, n number, node number) ? number
```

4. Exemples d'utilisation

```
import "std/c/graphcolor"  
use graphcolor  
  
create result = greedyColor(/* args */)   
say result  
  
create result = chromaticNumber(/* args */)   
say result  
  
create result = isValidColoring(/* args */)   
say result  
  
create result = colorClasses(/* args */)   
say result  
  
create result = conflictCount(/* args */)   
say result  
  
create result = welshPowell(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/c/graphcolor"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

module graphcolor

```
function greedyColor(adj list number, n number) returns list number
end

function chromaticNumber(adj list number, n number) returns number
end

function isValidColoring(adj list number, n number, colors list number) returns bool
end

function colorClasses(adj list number, n number) returns list number
end

function conflictCount(adj list number, n number, colors list number) returns number
end

function welshPowell(adj list number, n number) returns list number
end

function sameColorNeighbors(adj list number, n number, node number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphcolor` (tier complex, category complex). Exports 7 function(s): greedyColor, chromaticNumbe

```
`import "std/c/graphcolor"` . `use graphcolor`  
- `greedyColor(adj list number, n number) ? list number`  
- `chromaticNumber(adj list number, n number) ? number`  
- `isValidColoring(adj list number, n number, colors list number) ? bool`  
- `colorClasses(adj list number, n number) ? list number`  
- `conflictCount(adj list number, n number, colors list number) ? number`  
- `welshPowell(adj list number, n number) ? list number`  
- `sameColorNeighbors(adj list number, n number, node number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphcolor"  
use graphcolor
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? greedyColor(adj list number, n number) ? list  
? chromaticNumber(adj list number, n number) ? number  
? isValidColoring(adj list number, n number, colors list number) ? bool  
? colorClasses(adj list number, n number) ? list  
? conflictCount(adj list number, n number, colors list number) ? number  
? welshPowell(adj list number, n number) ? list  
? sameColorNeighbors(adj list number, n number, node number) ? number
```

4. Usage examples

```
import "std/c/graphcolor"  
use graphcolor  
  
create result = greedyColor(/* args */)   
say result  
  
create result = chromaticNumber(/* args */)   
say result  
  
create result = isValidColoring(/* args */)   
say result  
  
create result = colorClasses(/* args */)   
say result  
  
create result = conflictCount(/* args */)   
say result  
  
create result = welshPowell(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphcolor"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphcolor
```

```
function greedyColor(adj list number, n number) returns list number
end

function chromaticNumber(adj list number, n number) returns number
end

function isValidColoring(adj list number, n number, colors list number) returns bool
end

function colorClasses(adj list number, n number) returns list number
end

function conflictCount(adj list number, n number, colors list number) returns number
end

function welshPowell(adj list number, n number) returns list number
end

function sameColorNeighbors(adj list number, n number, node number) returns number
end
end
```